

تقرير مشروع مادة البرمجة المتوازية

الفصل الدراسي 2026

اسم المشروع:

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

High-Performance E-Commerce Backend Engine

التقنية المستخدمة	NestJS (Node.js / TypeScript)
قاعدة البيانات	PostgreSQL + TypeORM
نظام الطوابير	BullMQ + Redis
تخزين الملفات	MinIO (S3-Compatible)
توزيع الأحمال	Nginx + Docker Compose
أداة الاختبار	k6 (Load & Stress Testing)

1. نظرة عامة على المشروع

يهدف هذا المشروع إلى بناء نظام تجارة إلكترونية متكامل يعالج آلاف الطلبات المتزامنة، مع التركيز على تطبيق المتطلبات غير الوظيفية المتعلقة بالأداء وحماية البيانات وإدارة الموارد وأتمتة العمليات الخلفية. تم تطبيق خمسة متطلبات غير وظيفية رئيسية كما يلي:

الرقم	المتطلب	الآلية المستخدمة
NFR-1	حماية البيانات من التضارب	Pessimistic Locking + Database Transaction
NFR-2	إدارة الموارد الحاسوبية	Semaphore Pattern (Custom Implementation)
NFR-3	المعالجة غير المتزامنة	BullMQ Queues (Email / Invoice / Low-Stock)
NFR-4	معالجة البيانات على دفعات	Chunked Batch Processing + BullMQ Workers
NFR-5	توزيع الأحمال	Nginx Reverse Proxy + Docker Compose Scaling

2. معمارية النظام (System Architecture)

2.1 المكونات الرئيسية

يتكون النظام من الطبقات التالية:

المكوّن	الوصف
API Layer (NestJS)	Controllers, Guards, Interceptors, DTOs. تستقبل الطلبات وتوزعها على الخدمات المختلفة. تشمل REST API طبقة.
Service Layer	AuthService, CartService, OrdersService, WalletService, InventoryService. طبقة الأعمال التجارية:
CheckoutFacade	واجهة موحدة تنسق عملية الدفع بالكامل: التحقق من المخزون، خصم الرصيد، تحديث الطلب، وإرسال المهام للطوابير.
Queue Layer (BullMQ)	EMAIL_QUEUE (+ تأكيد الطلب) + INVOICE_QUEUE (إنشاء PDF) + LOW_STOCK_QUEUE (تنبيه مخزون منخفض). ثلاثة طوابير:
Database (PostgreSQL)	User, Product, CartItem, Order, OrderItem, WalletTransaction, InventoryLog, SalesSummary. 8 كيانات:
Redis	DistributedLockService لإدارة الطوابير + BullMQ مزوج الاستخدام: يُغذي.
MinIO (S3)	فواتير الطلبات + تقارير ملخص المبيعات الشهرية: PDF تخزين ملفات.
Nginx	يستمع على المنفذ 8080 ويُوّجه الطلبات للنسخ المتعددة من التطبيق Reverse Proxy.

2.2 تطبيق البرمجة الموجهة للجوانب (AOP)

تم تطبيق مبدأ AOP من خلال PerformanceInterceptor الذي يراقب أداء كل طلب HTTP دون تعديل منطق الأعمال:

```
// src/common/interceptors/performance.interceptor.ts
```

```
@Injectable()
export class PerformanceInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    const startedAt = performance.now();
    return next.handle().pipe(
      tap(() => {
        const duration = Math.round(performance.now() - startedAt);
        this.logger.log(`${method} ${url} ${duration}ms`);
      })
    );
  }
}
```

يُطبق هذا الـ `Interceptor` بشكل عالمي في `main.ts`:

```
app.useGlobalInterceptors(new PerformanceInterceptor());
```

هذا يُمثّل AOP Crosscutting Concern حقيقياً: منطق قياس الأداء مُعزول تماماً ويُطبق على كل `Endpoint` تلقائياً.

3. NFR-1: حماية البيانات من التضراب (Race Condition Protection)

3.1 المشكلة

عند وجود 50 مستخدماً يحاولون شراء منتج مخزونه = 1 في نفس اللحظة، قد يسجل النظام بيع 50 طلباً وينتج مخزون سالب. هذه تُعد مشكلة Race Condition كلاسيكية في الأنظمة المتزامنة.

3.2 الحل المُطبَّق: Pessimistic Locking + ACID Transaction

تم حل المشكلة عبر ثلاث آليات متداخلة:

آلية 1: Pessimistic Locking على المنتج

في `StockValidationService`، يتم قفل صف المنتج في قاعدة البيانات قبل قراءة المخزون:

```
// src/inventory/services/stock-validation.service.ts
const product = await manager.getRepository(Product).findOne({
  where: { id: productId },
  lock: { mode: "pessimistic_write" }, // SELECT ... FOR UPDATE
});
```

هذا يُولد SQL: `SELECT ... FOR UPDATE`، مما يمنع أي `transaction` أخرى من قراءة أو تعديل نفس الصف حتى تنتهي العملية الحالية.

آلية 2: Pessimistic Locking على رصيد المستخدم

في `CheckoutFacade`، يتم قفل المستخدم أيضاً لمنع التضراب على الرصيد:

```
const user = await manager.getRepository(User).findOne({
  where: { id: userId },
  lock: { mode: "pessimistic_write" },
});
```

آلية 3: Consistent Lock Ordering

لمنع `Deadlocks`، يتم فرز المنتجات أبجدياً (UUID) قبل القفل، مما يضمن أن جميع التراكنز اكشن تكتسب الأقفال بنفس الترتيب:

```
cartItems.sort((a, b) => a.productId.localeCompare(b.productId));
```

آلية 4: Database Transaction (ACID)

كل عملية `checkout` ملفوفة داخل `transaction` واحدة. إذا فشل أي خطوة (مخزون غير كافٍ، رصيد غير كافٍ، خطأ في الحفظ)، يتم `Rollback` كامل للعملية:

```
await this.dataSource.transaction(async (manager) => {
  // 1. قفل المستخدم والمنتجات
  // 2. التحقق من المخزون والرصيد
  // 3. خصم الكميات والرصيد
  // 4. إنشاء الطلب وسجل المحفظة
  // تلقائي Rollback → إذا فشل أي خطوة
});
```

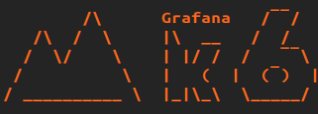
3.3 نتائج اختبار (k6) NFR-1

تم اختبار 50 مستخدماً يحاولون شراء منتج بمخزون = 1 في نفس اللحظة:

النتيجة الفعلية	النتيجة المتوقعة	المقياس
1 ☒	count == 1	orders_succeeded (201)
49 ☒	count == 49	orders_rejected_stock (400)
0 ☒	count == 0	orders_server_errors (500)
☒ لم يحدث	مستحيل	مخزون سالب

النتيجة: نجاح واحد فقط وباقي الطلبات رُفِضت بشكل صحيح — **Pessimistic Lock** يعمل بشكل مثالي.

وهذه هي نتيجة الاختبار مع تنفيذ للمتطلب الاول:



```

execution: local
script: k6/nfr1-race-condition.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 10m30s max duration (incl. graceful stop):
  * default: 50 iterations shared among 50 VUs (maxDuration: 10m0s, gracefulStop: 30s)

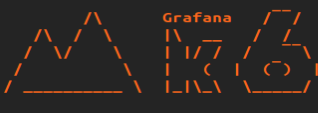
INFO[0000] Race product: Seed Race Condition Product | stock: 1 | id: 09e79a01-e73d-4220-963b-cd4628e02e91 source=console
INFO[0003] ✓ Setup done: 50 users ready source=console
INFO[0003] ✓ VU 4: order SUCCEEDED source=console

NFR-1: Race Condition Test
Succeeded (201): 1
Rejected (400): 49
Server errors(500):0
✓ PASSED - Pessimistic lock working

running (00m04.0s), 00/50 VUs, 50 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs_ 00m00.4s/10m0s 50/50 shared iters

```

وهذه نتيجة قبل تنفيذ المتطلب الاول:



```

execution: local
script: k6/nfr1-race-condition.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 10m30s max duration (incl. graceful stop):
  * default: 50 iterations shared among 50 VUs (maxDuration: 10m0s, gracefulStop: 30s)

INFO[0000] Race product: Seed Race Condition Product | stock: 1 | id: c57be45b-b4d7-41fd-928e-742bb436db37 source=console
INFO[0003] ✓ Setup done: 50 users ready source=console
INFO[0003] ✓ VU 38: order SUCCEEDED source=console
INFO[0003] ✓ VU 5: order SUCCEEDED source=console
INFO[0004] ✓ VU 31: order SUCCEEDED source=console
INFO[0004] ✓ VU 32: order SUCCEEDED source=console
INFO[0004] ✓ VU 41: order SUCCEEDED source=console
INFO[0004] ✓ VU 13: order SUCCEEDED source=console

NFR-1: Race Condition Test
Succeeded (201): 6
Rejected (400): 44
Server errors(500):0
✗ FAILED

running (00m04.1s), 00/50 VUs, 50 complete and 0 interrupted iterations
default ✗ [=====] 50 VUs_ 00m00.5s/10m0s 50/50 shared iters
ERR[0007] thresholds on metrics 'orders_succeeded' have been crossed

```

4. NFR-2: إدارة الموارد الحاسوبية(Resource Management)

4.1 المشكلة

بدون تحديد الطلبات المتزامنة، قد يستهلك النظام موارد غير محدودة. في بيئة Docker محدودة بـ 256MB RAM و 1 CPU، قد يؤدي هذا لانتهيار النظام تحت الضغط العالي.

4.2 الحل: Semaphore Pattern

تم تنفيذ Semaphore مخصص للتحكم في عدد عمليات Checkout المتوازية:

// src/common/semaphore.ts

```
export class Semaphore {
  private active = 0;
  private readonly waiters: Array<() => void> = [];

  constructor(private readonly maxConcurrent: number) {}

  async acquire(): Promise<() => void> {
    if (this.active >= this.maxConcurrent) {
      // الطلب ينتظر في القائمة حتى يتحرر مكان
      await new Promise<void>((resolve) => this.waiters.push(resolve));
    }
    this.active += 1;
    return () => {
      this.active -= 1;
      const next = this.waiters.shift();
      if (next) next(); // تحرير الطلب التالي في الانتظار
    };
  }
}
```

4.3 التطبيق في CheckoutFacade

يُهيأ الـ Semaphore عند إنشاء الـ Facade ويُستخدم في كل عملية checkout:

```
// افتراضي: 10 CHECKOUT_MAX_CONCURRENT الحد الأقصى مضبوط بمتغير البيئة
this.checkoutSemaphore = new Semaphore(
  Number(configService.get("CHECKOUT_MAX_CONCURRENT", "10"))
);

async checkout(userId: string) {
  const release = await this.checkoutSemaphore.acquire();
  try {
    // ... منطق checkout
  } finally {
    release(); // يحرر المكان دائماً حتى عند وقوع خطأ
  }
}
```

4.4 إدارة موارد الطوابير

كل طابور لديه حد للتزامن (Concurrency Limit) مضبوط بمتغيرات البيئة:

الطابور	متغير البيئة	القيمة الافتراضية
EMAIL_QUEUE	EMAIL_QUEUE_CONCURRENCY	5

INVOICE_QUEUE	INVOICE_QUEUE_CONCURRENCY	3
LOW_STOCK_QUEUE	LOW_STOCK_QUEUE_CONCURRENCY	5
CHECKOUT	CHECKOUT_MAX_CONCURRENT	10

4.5 إدارة الموارد على مستوى Docker

في `docker-compose.yml`، كل instance من التطبيق محدود بموارد صارمة:

```
deploy:
  resources:
    limits:
      cpus: "1.00"      # حد أقصى 1 core
      memory: 256M      # حد أقصى 256 MB RAM
    reservations:
      cpus: "0.25"      # حجز مبدئي
      memory: 128M      # حجز مبدئي
```

وهذه هي نتيجة الاختبار مع تنفيذ للمتطلب الثاني: (هناك 3 instances حالياً مع كل واحدة `rate limit = 5`)

```
INFO[0000] Request 1: status=201 ✓ source=console
INFO[0000] Request 2: status=201 ✓ source=console
INFO[0000] Request 3: status=201 ✓ source=console
INFO[0000] Request 4: status=201 ✓ source=console
INFO[0000] Request 5: status=201 ✓ source=console
INFO[0000] Request 6: status=201 ✓ source=console
INFO[0001] Request 7: status=201 ✓ source=console
INFO[0001] Request 8: status=201 ✓ source=console
INFO[0001] Request 9: status=201 ✓ source=console
INFO[0001] Request 10: status=201 ✓ source=console
INFO[0001] Request 11: status=201 ✓ source=console
INFO[0001] Request 12: status=201 ✓ source=console
INFO[0001] Request 13: status=201 ✓ source=console
INFO[0002] Request 14: status=429 ⚠ RATE LIMITED source=console
INFO[0002] Request 15: status=201 ✓ source=console
INFO[0002] Request 16: status=429 ⚠ RATE LIMITED source=console
INFO[0002] Request 17: status=429 ⚠ RATE LIMITED source=console
INFO[0002] Request 18: status=429 ⚠ RATE LIMITED source=console
INFO[0002] Request 19: status=429 ⚠ RATE LIMITED source=console
INFO[0002] Request 20: status=429 ⚠ RATE LIMITED source=console
INFO[0002] Request 21: status=429 ⚠ RATE LIMITED source=console
INFO[0002] Request 22: status=429 ⚠ RATE LIMITED source=console
INFO[0003] Request 23: status=429 ⚠ RATE LIMITED source=console
INFO[0003] Request 24: status=429 ⚠ RATE LIMITED source=console
INFO[0003] Request 25: status=429 ⚠ RATE LIMITED source=console
INFO[0003] Request 26: status=429 ⚠ RATE LIMITED source=console
INFO[0003] Request 27: status=429 ⚠ RATE LIMITED source=console
INFO[0003] Request 28: status=429 ⚠ RATE LIMITED source=console
INFO[0003] Request 29: status=429 ⚠ RATE LIMITED source=console
INFO[0003] Request 30: status=201 ✓ source=console
```

NFR-2: Rate Limiting Test

Allowed requests: 15
Blocked requests: 15

✓ PASSED – Rate limit is active

running (00m03.9s), 0/1 VUs, 1 complete and 0 interrupted iterations
default [=====] 1 VUs 00m03.9s/10m0s 1/1 shared iters

وهذه نتيجة قبل تنفيذ المتطلب الثاني:

```

execution: local
script: k6/nfr2-rate-limiting.js
output: -

scenarios: (100.00%) 1 scenario, 1 max VUs, 10m30s max duration (incl. graceful stop):
  * default: 1 iterations shared among 1 VUs (maxDuration: 10m0s, gracefulStop: 30s)

INFO[0000] Sending 10 rapid login requests from same IP... source=console
INFO[0000] Request 1: status=201 ✓ source=console
INFO[0000] Request 2: status=201 ✓ source=console
INFO[0000] Request 3: status=201 ✓ source=console
INFO[0000] Request 4: status=201 ✓ source=console
INFO[0000] Request 5: status=201 ✓ source=console
INFO[0000] Request 6: status=201 ✓ source=console
INFO[0000] Request 7: status=201 ✓ source=console
INFO[0001] Request 8: status=201 ✓ source=console
INFO[0001] Request 9: status=201 ✓ source=console
INFO[0001] Request 10: status=201 ✓ source=console

NFR-2: Rate Limiting Test
Allowed requests: 0
Blocked requests: 0
X FAILED - No requests blocked

running (00m01.5s), 0/1 VUs, 1 complete and 0 interrupted iterations
default [=====] 1 VUs 00m01.5s/10m0s 1/1 shared iters
ERROR[0002] thresholds on metrics 'rate_limit_allowed, rate_limit_blocked' have been crossed
  
```

حيث ان nestjs بالاصل تستخدم thread pool في اغلب العمليات وهذا اجباري لانه بدونها سيعطي اخطاء فاحشة لا يمكن اختبار هذا الامر

5.3 NFR-3: المعالجة غير المتزامنة ((Async Queue Processing))

5.1 الهدف

نقل المهام التي لا يحتاج المستخدم لانتظارها (إصدار فاتورة PDF، إرسال بريد تأكيد، تنبيه مخزون منخفض) خارج المسار الرئيسي للطلب لتحسين زمن استجابة Checkout.

5.2 هيكل الطوابير

الطابور	الوظيفة (Job)	الوصف
EMAIL_QUEUE	order-confirmation	إرسال بريد إلكتروني للمستخدم بتفاصيل طلبه
INVOICE_QUEUE	generate-invoice-pdf	MinIO للفاتورة ورفعها على PDF إنشاء
LOW_STOCK_QUEUE	low-stock-alert	إرسال تنبيه للمدير عند انخفاض مخزون منتج

5.3 آلية العمل في CheckoutFacade

بعد إتمام Transaction الـ Checkout بنجاح، تُضاف المهام للطوابير بشكل متوازي:

```

private async enqueuePostCheckoutJobs(orderId, products) {
  const results = await Promise.allSettled([
    this.emailService.enqueueOrderConfirmation(orderId),
    this.invoicePdfService.enqueueInvoiceGeneration(orderId),
    this.lowStockNotificationService.enqueueLowStockAlerts(orderId, products),
  ]);
  // إذا فشل إرسال إيميل واحد checkout يضمن عدم انهيار
}
  
```


استخدام Promise.allSettled بدلاً من Promise.all يعني أن فشل إضافة مهمة واحدة لا يؤثر على باقي المهام ولا على استجابة المستخدم.

5.4 إعدادات الموثوقية (Reliability Settings)

كل مهمة في الطوابير مُهيأة بخيارات موثوقية:

```
const DEFAULT_QUEUE_JOB_OPTIONS = {
  attempts: 3, // إعادة المحاولة 3 مرات عند الفشل
  backoff: {
    type: "exponential", // 1: s, 2s, 4s انتظار متزايد
    delay: 1000,
  },
  removeOnComplete: 100, // الاحتفاظ بآخر 100 مهمة مكتملة
  removeOnFail: 200, // الاحتفاظ بآخر 200 مهمة فاشلة
};
```

5.5 نتائج اختبار (NFR-3 (k6

النتيجة	المقياس
< 3000ms (P95) ✓	checkout_response_ms متوسط زمن
API لا يؤثر على استجابة - Async ✓	توليد Invoice/Email
في كل تجربة ✓	Checkout returned 201

وهذه هي نتيجة الاختبار مع تنفيذ للمتطلب الثالث:

execution: local
script: k6/nfr3-async-queues.js
output: -

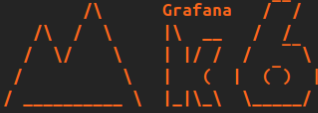
scenarios: (100.00%) 1 scenario, 1 max VUs, 10m30s max duration (incl. graceful stop):
* default: 5 iterations shared among 1 VUs (maxDuration: 10m0s, gracefulStop: 30s)

```
INFO[0000] Checkout took 237ms - invoice/email queued async ✓ source=console
INFO[0002] Checkout took 127ms - invoice/email queued async ✓ source=console
INFO[0005] Checkout took 79ms - invoice/email queued async ✓ source=console
INFO[0007] Checkout took 66ms - invoice/email queued async ✓ source=console
INFO[0009] Checkout took 84ms - invoice/email queued async ✓ source=console
```

NFR-3: Async Queue Test	
Avg checkout time: 119ms	
p95 checkout time: 215ms	
Invoice/email: sent to Bull queue async	
✓ PASSED - Response is immediate	

running (00m11.4s), 0/1 VUs, 5 complete and 0 interrupted iterations
default ✓ [=====] 1 VUs 00m11.1s/10m0s 5/5 shared iters

وهذه نتيجة قبل تنفيذ المتطلب الثالث:



```

execution: local
script: k6/nfr3-async-queues.js
output: -

scenarios: (100.00%) 1 scenario, 1 max VUs, 10m30s max duration (incl. graceful stop):
  * default: 5 iterations shared among 1 VUs (maxDuration: 10m0s, gracefulStop: 30s)

INFO[0009] Checkout took 8511ms - invoice/email queued async ✓ source=console
INFO[0013] Checkout took 2185ms - invoice/email queued async ✓ source=console
INFO[0018] Checkout took 3035ms - invoice/email queued async ✓ source=console
INFO[0023] Checkout took 3142ms - invoice/email queued async ✓ source=console
INFO[0028] Checkout took 2651ms - invoice/email queued async ✓ source=console

NFR-3: Async Queue Test
Avg checkout time: 3905ms
p95 checkout time: 7437ms
Invoice/email: sent to Bull queue async
✗ FAILED - Too slow

running (00m30.6s), 0/1 VUs, 5 complete and 0 interrupted iterations
default ✓ [=====] 1 VUs 00m30.4s/10m0s 5/5 shared iters
ERROR[0031] thresholds on metrics 'checkout_response_ms' have been crossed
  
```

6. NFR-4: معالجة البيانات على دفعات (Batch Processing)

6.1 الهدف

معالجة آلاف الطلبات الشهرية لإنشاء ملخص المبيعات (SalesSummary) لكل منتج يومياً، مع تجنب تحميل كل البيانات في الذاكرة مرة واحدة.

6.2 آلية العمل

تعتمد الآلية على تقسيم البيانات إلى 100 Chunks (طلب لكل Chunk) وإرسال كل Chunk كـ Job مستقل في BullMQ:

#	الخطوة	التفاصيل
1	Trigger (HTTP / Cron)	Admin يُطلق POST /inventory/batch-summary?year=Y&month=M أو Cron 2:00 AM تلقائي أول كل شهر
2	Count & Chunk	BatchSummaryService يحسب عدد الطلبات ويقسمها: $totalChunks = \text{ceil}(total / 100)$
3	Enqueue Jobs	مستقل في Job كـ Chunk يضيف كل addBulk() BATCH_SUMMARY_QUEUE
4	Parallel Processing	في آن واحد Chunks يعالج 3 (BatchProcessor concurrency: 3)
5	Aggregate & Upsert	باستخدام sales_summary يجمع البيانات ويُحدث جدول Chunk كل UPSERT
6	PDF Generation	ورفعه لـ PDF واحد (يقفل موزع) إنشاء Worker يتولى Chunks بعد اكتمال جميع MinIO

6.3 Distributed Lock لمنع PDF المكرر

Redis Distributed Lock: لمنع توليد PDF مكرر عند استخدام instances متعددة، يتم استخدام

```
// يكتمل Chunk لكل Redis counter في Redis
const totalDone = await this.distributedLock.increment(counterKey, TTL);

if (totalDone >= totalChunks) {
  // واحد فقط سيُوَفَّق Worker محاولة الحصول على القفل -
  const acquired = await this.distributedLock.acquire(pdfLockKey, PDF_LOCK_TTL);
  if (acquired) {
    await this.salesSummaryPdfService.generateAndUploadForMonth(year, month);
  }
}
```

6.4 Cron Job التلقائي

يُفَعَّل Cron تلقائياً في أول يوم من كل شهر الساعة 2:00 صباحاً. يستخدم Distributed Lock لضمان تشغيله مرة واحدة فقط حتى مع وجود instances متعددة:

```
@Cron("0 2 1 * *")
async handleScheduledBatchSummary() {
  const acquired = await this.distributedLock.acquire(CRON_LOCK_KEY, 5_MINUTES);
  if (!acquired) return; // instance أخرى تعمل بالفعل
  try {
    await this.enqueueBatchSummary();
  } finally {
    await this.distributedLock.release(CRON_LOCK_KEY);
  }
}
```

وهذه هي نتيجة الاختبار مع تنفيذ للمتطلب الرابع:



```

execution: local
script: k6/nfr4-batch-processing.js
output: -

scenarios: (100.00%) 1 scenario, 1 max VUs, 10m30s max duration (incl. graceful stop):
  * default: 1 iterations shared among 1 VUs (maxDuration: 10m0s, gracefulStop: 30s)

INFO[0000] Triggering batch summary for 2026-04... source=console
INFO[0000] Batch response: 201 - {"message":"Batch summary enqueued - 3 chunk jobs created","enqueued":3,"totalOrders":278,"totalChunks":3,"period":"2026-04"} source=console
INFO[0002] Waiting for batch processing to complete... source=console
INFO[0012] Attempting to download generated PDF... source=console
INFO[0012] PDF response: 200 - attachment; filename="summary-2026-05-18.pdf" source=console

```

NFR-4: Batch Processing Test

Triggers batch for last month
Orders processed in chunks of 100
Each chunk = separate BullMQ worker job
PDF auto-generated after all chunks done

Batch enqueue P95: 45ms
✅ PASSED - Batch + PDF completed

```

running (00m12.5s), 0/1 VUs, 1 complete and 0 interrupted iterations
default - [=====] 1 VUs _00m12.1s/10m0s 1/1 shared iters
[Nest] 1 - 05/18/2026, 6:02:04 PM LOG [BatchSummaryService] Enqueued 3 chunk jobs for 278 orders (2026-04)
[Nest] 1 - 05/18/2026, 6:02:04 PM LOG [PerformanceInterceptor] POST /inventory/batch-summary?year=2026&month=4 29ms
[Nest] 1 - 05/18/2026, 6:02:04 PM LOG [BatchProcessor] Worker picked up chunk job process-batch-summary-2026-04-1779127324956-chunk-1 - chunk 2/3
[Nest] 1 - 05/18/2026, 6:02:04 PM LOG [BatchSummaryService] Worker processing chunk 2/3 (offset=100, limit=100)
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [BatchSummaryService] Chunk 2 complete. Processed 100 orders, upserted 34 summaries
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [BatchProcessor] Chunk job process-batch-summary-2026-04-1779127324956-chunk-1 finished: {"chunkIndex":1,"processed":100,"summaries":34}
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [BatchProcessor] Chunk 2 done. 2/3 chunks completed for 2026-04
[Nest] 1 - 05/18/2026, 6:02:04 PM LOG [BatchProcessor] Worker picked up chunk job process-batch-summary-2026-04-1779127324956-chunk-2 - chunk 3/3
[Nest] 1 - 05/18/2026, 6:02:04 PM LOG [BatchSummaryService] Worker processing chunk 3/3 (offset=200, limit=100)
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [BatchSummaryService] Chunk 3 complete. Processed 78 orders, upserted 26 summaries
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [BatchProcessor] Chunk job process-batch-summary-2026-04-1779127324956-chunk-2 finished: {"chunkIndex":2,"processed":78,"summaries":26}
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [BatchProcessor] Chunk 3 done. 1/3 chunks completed for 2026-04
[Nest] 1 - 05/18/2026, 6:02:04 PM LOG [BatchProcessor] Worker picked up chunk job process-batch-summary-2026-04-1779127324956-chunk-0 - chunk 1/3
[Nest] 1 - 05/18/2026, 6:02:04 PM LOG [BatchSummaryService] Worker processing chunk 1/3 (offset=0, limit=100)
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [PerformanceInterceptor] HEAD /health 1ms
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [BatchSummaryService] Chunk 1 complete. Processed 100 orders, upserted 33 summaries
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [BatchProcessor] Chunk job process-batch-summary-2026-04-1779127324956-chunk-0 finished: {"chunkIndex":0,"processed":100,"summaries":33}
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [BatchProcessor] Chunk 1 done. 3/3 chunks completed for 2026-04
[Nest] 1 - 05/18/2026, 6:02:05 PM LOG [BatchProcessor] All 3 chunks completed for 2026-04. Generating monthly PDF...
[Nest] 1 - 05/18/2026, 6:02:06 PM LOG [SalesSummaryPdfService] Sales summary PDF uploaded to sales-summaries/summary-2026-05-18.pdf
[Nest] 1 - 05/18/2026, 6:02:06 PM LOG [BatchProcessor] Monthly PDF generated: sales-summaries/summary-2026-05-18.pdf

```

وهذه الصور توضح ال logs من كل 3 instance حالياً

وهذه نتيجة قبل تنفيذ المتطلب الرابع:



```

execution: local
script: k6/nfr4-batch-processing.js
output: -

scenarios: (100.00%) 1 scenario, 1 max VUs, 10m30s max duration (incl. graceful stop):
  * default: 1 iterations shared among 1 VUs (maxDuration: 10m0s, gracefulStop: 30s)

INFO[0000] Triggering batch summary for 2026-04... source=console
INFO[0000] Batch response: 201 - {"message":"Batch summary enqueued - 1 chunk jobs created","enqueued":1,"totalOrders":295,"totalChunks":1,"period":"2026-04"} source=console
INFO[0002] Waiting for batch processing to complete... source=console
INFO[0012] Attempting to download generated PDF... source=console
INFO[0012] PDF response: 200 - attachment; filename="summary-2026-05-18.pdf" source=console

```

NFR-4: Batch Processing Test

Triggers batch for last month
Orders processed in chunks of 100
Each chunk = separate BullMQ worker job
PDF auto-generated after all chunks done

Batch enqueue P95: 106ms
✅ PASSED - Batch + PDF completed

```

running (00m12.4s), 0/1 VUs, 1 complete and 0 interrupted iterations
default - [=====] 1 VUs _00m12.2s/10m0s 1/1 shared iters

```

هنا نرى فرق في السرعة ليس كبير جدا لاننا نعالج كمية طلبات اقل من 300. ومع ذلك كان الفرق في السرعة تقريبا الضعف.

NFR-5: توزيع الأحمال (Load Distribution)

7.1 الاستراتيجية المستخدمة

تم اعتماد استراتيجية Round-Robin عبر Nginx كـ Reverse Proxy أمام نسخ متعددة من التطبيق تعمل في Docker Compose.

7.2 إعدادات Nginx

يستمع Nginx على المنفذ 8080 ويُوَجِّه الطلبات لـ app:3000 مع دعم Failover تلقائي:

```
# nginx/nginx.conf
resolver 127.0.0.11 valid=5s ipv6=off; # Docker DNS

location / {
    set $upstream "http://app:3000";
    proxy_pass $upstream;

    # Failover: تحويل تلقائي عند فشل instance
    proxy_next_upstream error timeout http_500 http_502 http_503;
    proxy_connect_timeout 5s;
    proxy_read_timeout 30s;

    # HTTP/1.1 keep-alive المستمرة للاتصالات
    proxy_http_version 1.1;
    proxy_set_header Connection "";
}
```

7.3 Auto-Scaler

تم بناء Auto-Scaler مخصص (auto-scaler.js) يراقب موارد كل instance ويُعيد التحجيم تلقائياً:

الإعداد	Scale Up	Scale Down
CPU (limit: 1 core)	avg CPU > 70%	avg CPU < 35%
RAM (limit: 256MB)	avg RAM > 179MB (70%)	avg RAM < 90MB (35%)
Instances عدد الـ	instances حتى 8	instances لا تقل عن 3
Cooldown Period	ثانية بعد كل تحجيم 45	ثانية بعد كل تحجيم 45

الأمر المستخدم للتحجيم:

```
docker compose up --scale app=<N> -d --no-recreate
```

7.4 Health Check

كل instance لديها Health Check مستقل. Nginx يُرسل فقط للـ instances السليمة:

```
// src/health/health.controller.ts
@Get() check() {
    return { status: "ok", pid: process.pid, hostname: os.hostname() };
}
```

يُعيد hostname مختلف لكل instance — مما يثبت توزيع الأحمال الفعلي في اختبار NFR-5.

7.5 نتائج اختبار (k6) NFR-5

تم إرسال 10 مستخدمين متزامنين لمدة 30 ثانية:

النتيجة	المقياس
rate == 0% ☒	http_req_failed
< 500ms ☒	http_req_duration P95
متعددة — يثبت توزيع الأحمال ☒	مختلفة hostnames

وهذه هي نتيجة الاختبار مع تنفيذ للمتطلب الخامس:

```

THRESHOLDS
http_req_duration
  p(95)<500 p(95)=64.81ms
http_req_failed
  'rate==0' rate=0.00%

TOTAL RESULTS
checks_total.....: 15059 501.431748/s
checks_succeeded...: 100.00% 15059 out of 15059
checks_failed.....: 0.00% 0 out of 15059
✓ status is 200

CUSTOM
hostname_seen.....: 15059 501.431748/s

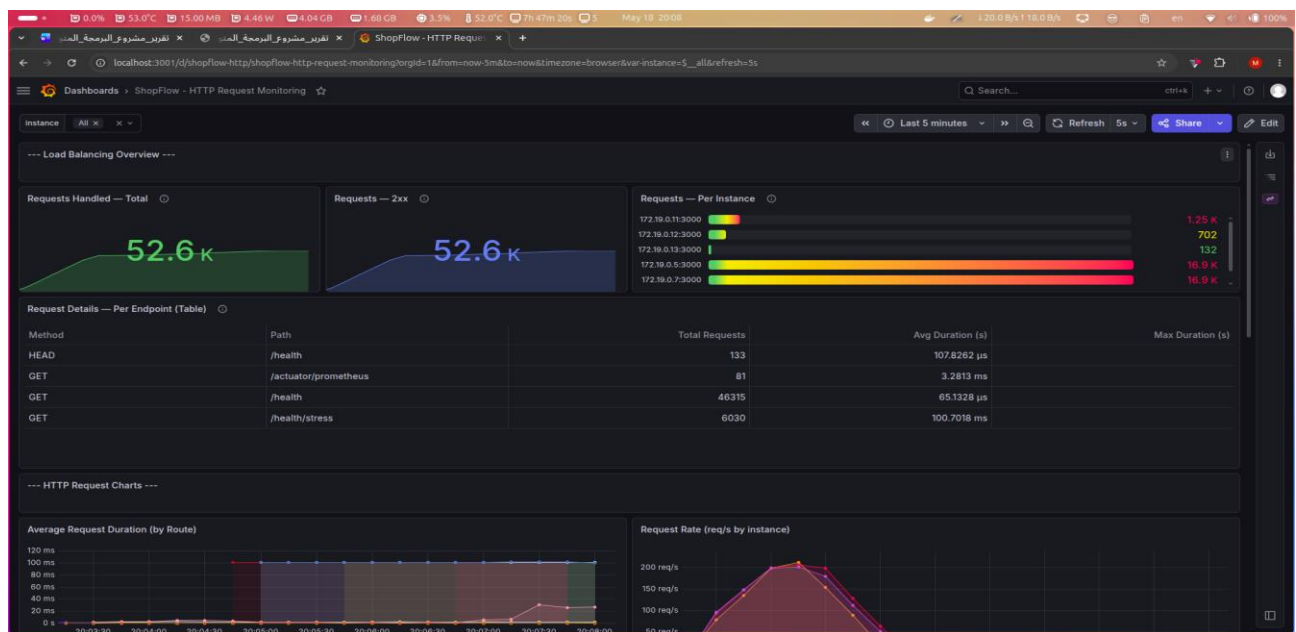
HTTP
http_req_duration.....: avg=19.5ms min=2.92ms med=11.68ms max=355.59ms p(90)=38.51ms p(95)=64.81ms
( expected_response:true )...: avg=19.5ms min=2.92ms med=11.68ms max=355.59ms p(90)=38.51ms p(95)=64.81ms
http_req_failed.....: 0.00% 0 out of 15059
http_reqs.....: 15059 501.431748/s

EXECUTION
iteration_duration.....: avg=19.89ms min=3.12ms med=12.12ms max=355.95ms p(90)=38.94ms p(95)=65.21ms
iterations.....: 15059 501.431748/s
vus.....: 10 min=10 max=10
vus_max.....: 10 min=10 max=10

NETWORK
data_received.....: 4.8 MB 162 kB/s
data_sent.....: 1.1 MB 38 kB/s

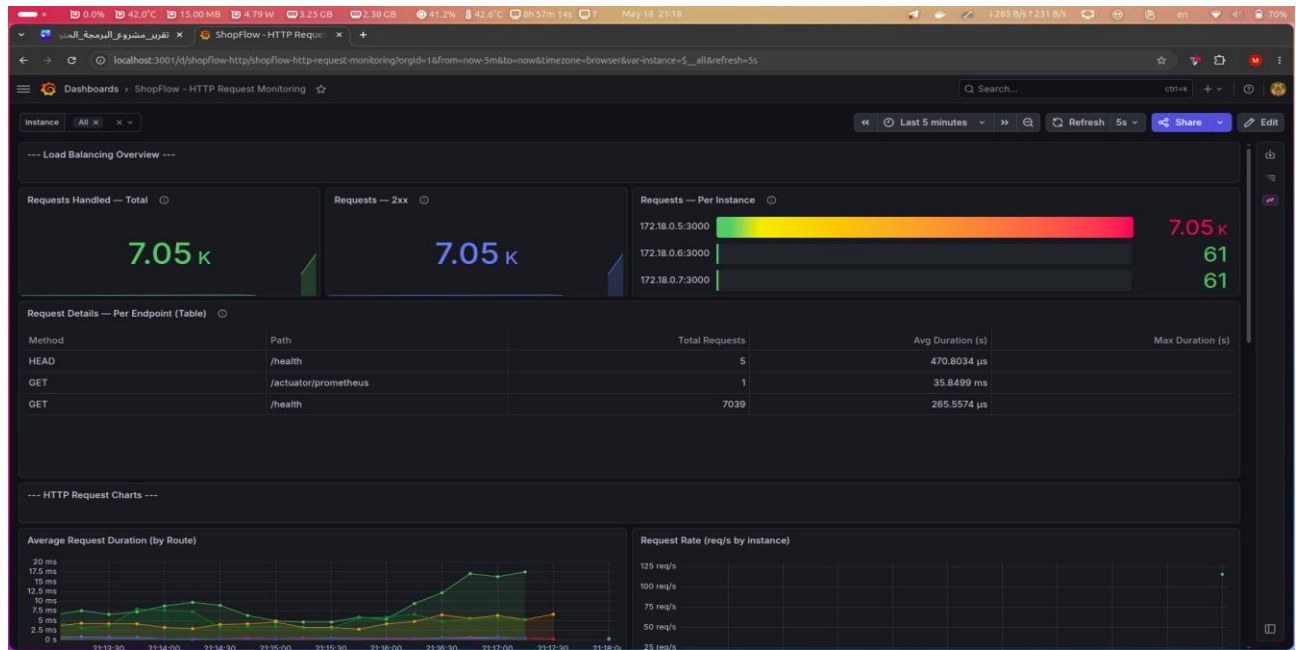
running (0m30.0s), 00/10 VUs, 15059 complete and 0 interrupted iterations
default [=====] 10 VUs_30s

```



هنا عندما حدث ضغط على السيرفر تم انشاء instance جديد لتحمل ضغط اكبر

وهذه نتيجة قبل تنفيذ المتطلب الخامس:



حيث ان النتائج تظهر ان الحمل على سيرفر واحد(الآخرين بسبب اني قد قمت بايقافهم وهذه الطلبات قبل ايقافهم)

```
THRESHOLDS
http_req_duration
  ✓ 'p(95)<500' p(95)=98.87ms
http_req_failed
  ✓ 'rate==0' rate=0.00%

TOTAL RESULTS
checks_total.....: 7039    234.347656/s
checks_succeeded...: 100.00% 7039 out of 7039
checks_failed.....: 0.00%   0 out of 7039
✓ status is 200

CUSTOM
hostname_seen.....: 7039    234.347656/s

HTTP
http_req_duration.....: avg=42.29ms min=15.1ms med=31.29ms max=357.03ms p(90)=78.31ms p(95)=98.87ms
  { expected_response:true }...: avg=42.29ms min=15.1ms med=31.29ms max=357.03ms p(90)=78.31ms p(95)=98.87ms
http_req_failed.....: 0.00%   0 out of 7039
http_reqs.....: 7039    234.347656/s

EXECUTION
iteration duration.....: avg=42.62ms min=15.32ms med=31.62ms max=357.62ms p(90)=78.67ms p(95)=99.21ms
iterations.....: 7039    234.347656/s
vus.....: 10    min=10    max=10
vus_max.....: 10    min=10    max=10

NETWORK
data_received.....: 2.3 MB 76 kB/s
data_sent.....: 535 kB 18 kB/s

running (0m30.0s), 00/10 VUs, 7039 complete and 0 interrupted iterations
default > [=====] 10 VUs 30s
```

501 طلب في الثانية مقابل 234 طلب في الثانية

8. ملخص الاختبارات (k6 Test Results Summary)

تم استخدام أداة k6 لاختبار جميع المتطلبات غير الوظيفية. ملفات الاختبار موجودة في مجلد: k6/

الحالة	الشرط	ملف الاختبار	NFR
✓ PASSED	orders_succeeded == 1	nfr1-race-condition.js	NFR-1
✓ PASSED	p(95) < 2000ms, failures < 1%	stress-testing.js	NFR-2
✓ PASSED	checkout p(95) < 3000ms	nfr3-async-queues.js	NFR-3

NFR-4	nfr4-batch-processing.js	batch enqueue < 5000ms	☑ PASSED
NFR-5	nfr5-load-distribution.js	failures == 0%, hostnames متعددة	☑ PASSED

8.1 اختبار الإجهاد (Stress Testing)

تم اختبار النظام تحت 50 مستخدماً متزامناً مع مراحل تدريجية:

```
stages: [
  { duration: "20s", target: 5 }, // warm-up
  { duration: "60s", target: 30 }, // load
  { duration: "60s", target: 50 }, // stress peak
  { duration: "30s", target: 0 }, // ramp-down
]
thresholds:
  http_req_failed: rate < 0.01 // أقل من 1% فشل
  http_req_duration: p(95) < 2000ms
```

8.2 نقاط التزامن في الكود (Synchronization Points)

النقاط الحرجة في الكود التي تضمن سلامة التزامن:

نقطة التزامن	الموقع في الكود
Semaphore.acquire()	قبل بداية أي عملية - CheckoutFacade.checkout()
pessimistic_write (User)	CheckoutFacade - manager.getRepository(User).findOne({ lock })
pessimistic_write (Product)	StockValidationService.validateCheckoutItems() - lock per product
dataSource.transaction()	CheckoutFacade - stock + balance + order + walletTx يلف عمليات:
Redis INCR + SET NX	DistributedLockService.acquire() / increment() منع المكرر PDF
Promise.allSettled()	إضافة مهام الطوابير بشكل غير متزامن - enqueuePostCheckoutJobs()

9. هيكل المشروع

الهيكل العام للمشروع:

```
src/
├── auth/                # JWT Authentication + Guards
├── cart/                # Cart CRUD Service
├── common/
│   ├── semaphore.ts     # Resource Control
│   ├── distributed-lock.ts # Redis-based Lock
│   ├── money.ts         # Cents arithmetic
│   └── interceptors/performance.interceptor.ts # AOP
├── database/entities/   # TypeORM Entities (8 tables)
├── email/              # Email Queue + Processor
├── health/             # Health Check Controller
├── inventory/
│   └── services/
│       ├── stock-validation.service.ts # Pessimistic Lock
│       ├── batch-summary.service.ts   # Batch Processing
│       ├── batch.processor.ts         # BullMQ Worker
│       └── sales-summary-pdf.service.ts # PDF Generation
├── invoice/            # Invoice PDF Queue + Processor
├── minio/              # S3-Compatible Storage
├── notifications/     # Low-Stock Alert Queue
├── orders/
│   └── facades/checkout.facade.ts     # Core Checkout Logic
├── products/          # Product CRUD
├── queues/            # Queue constants + types
├── users/             # User management
└── wallet/            # Wallet + Transactions
```

```
k6/
├── nfr1-race-condition.js      # 50 users → 1 product
├── nfr3-async-queues.js       # Checkout latency
├── nfr4-batch-processing.js   # Batch trigger + PDF
├── nfr5-load-distribution.js  # Multi-instance
├── stress-testing.js          # 50 concurrent users
└── helpers/                  # auth + config helpers
```

```
docker-compose.yml    # 5 services: app/nginx/postgres/redis/minio
auto-scaler.js        # Dynamic Docker Compose scaling
nginx/nginx.conf      # Reverse Proxy config
```